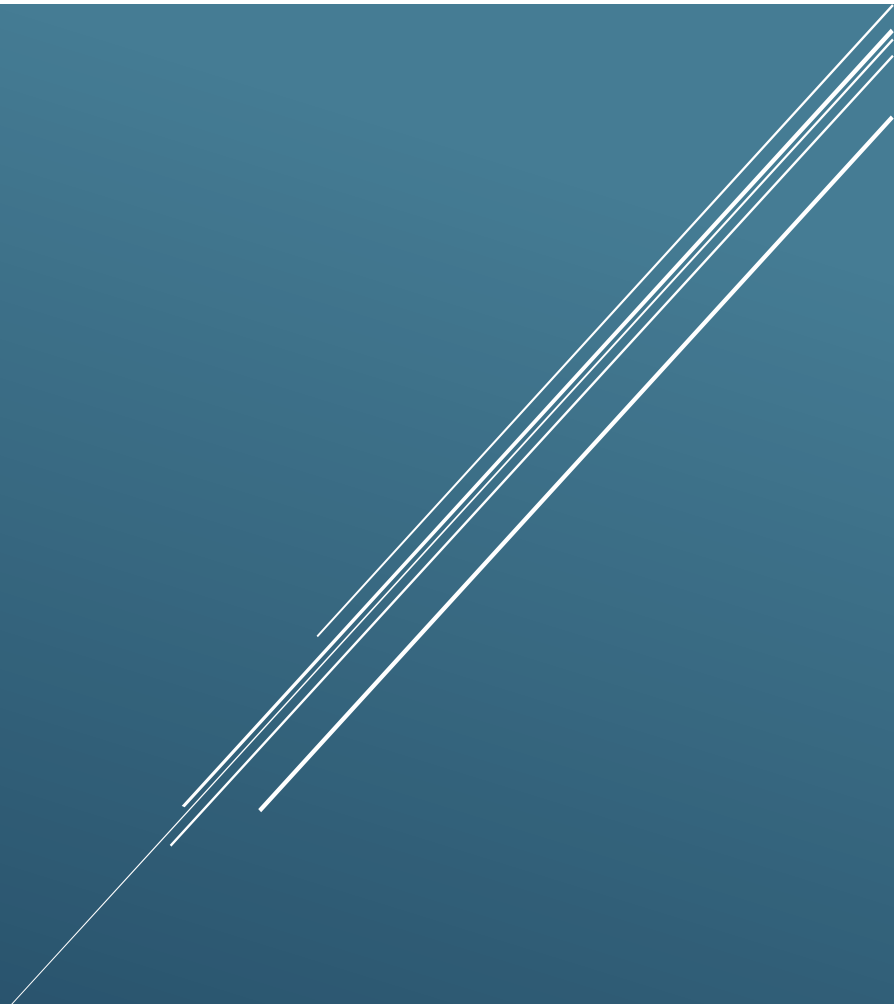




# PERRERA AMIGUITOS

Proyecto TABD

## Contenido

|     |                                                                                |   |
|-----|--------------------------------------------------------------------------------|---|
| 1.  | Introducción al documento .....                                                | 2 |
| 2.  | Descripción del contexto y de los requisitos funcionales y no funcionales..... | 3 |
| 2.1 | Introducción .....                                                             | 3 |
| 2.2 | Requisitos Funcionales (RF) .....                                              | 3 |
| 2.3 | Requisitos No Funcionales (RNF) .....                                          | 4 |
| 3.  | Diagrama de clases persistentes. ....                                          | 5 |
| 4.  | Esquema lógico específico O/R.....                                             | 6 |
| 5.  | Diseño Físico. ....                                                            | 6 |
| 6.  | Disparadores y paquete .....                                                   | 7 |
| 6.1 | Paquete funcionesRefugio .....                                                 | 7 |
| 6.2 | Disparadores (Triggers) .....                                                  | 8 |
| A.  | Trigger_SuministrarEsenciales.....                                             | 8 |
| B.  | Trigger_AsignarCuotas .....                                                    | 8 |
| 7.  | Conclusiones.....                                                              | 9 |

# 1. Introducción al documento

El presente documento constituye la memoria técnica del sistema de gestión para el refugio de animales, un proyecto diseñado bajo el paradigma Objeto-Relacional. El objetivo principal de este informe es detallar cómo se ha transformado una necesidad administrativa y sanitaria en una arquitectura de datos robusta, automatizada y escalable en un entorno Oracle.

A lo largo de las siguientes secciones, encontraras un desglose completo del ciclo de vida del desarrollo:

- **Análisis de Requisitos:** En primer lugar, se definen las necesidades funcionales, centrándose en la automatización de los protocolos de salud y el control de socios.
- **Diseño de la Arquitectura:** Se presenta el **Diagrama de Clases**, que sirve como base conceptual, seguido del **Esquema Lógico O/R**, donde se explica el observa la relaciones entre objetos.
- **Implementación Técnica:** Se detalla el **Diseño Físico** de las tablas y la lógica de programación encapsulada en el paquete funcionesRefugio. En este apartado cobran especial importancia los **Disparadores (Triggers)**, piezas clave que garantizan la integridad de los datos y ejecutan acciones automáticas tras cada ingreso o actualización.
- **Validación y Despliegue:** El documento concluye con un **Manual de Instalación** paso a paso para la puesta en marcha del sistema y una carga de datos realista que demuestra el comportamiento del software tras un año simulado de actividad.

## 2. Descripción del contexto y de los requisitos funcionales y no funcionales.

### 2.1 Introducción

El sistema gestiona un **Refugio de Animales** multiespecie (perros, gatos o lo que necesite el refugio) cuya misión es la acogida, el cuidado sanitario y la búsqueda de un nuevo hogar para animales en situación de abandono. El funcionamiento del centro se apoya en tres pilares fundamentales:

1. **Bienestar Animal y Salud:** Ante el desconocimiento del historial previo de los animales al ingresar, el refugio aplica un protocolo sanitario riguroso. Esto incluye una **vacunación inicial automática** de todas las vacunas esenciales y el registro de dosis de refuerzo o vacunas opcionales administradas por facultativos veterinarios.
2. **Gestión Flexible de Adopciones:** A diferencia de otros centros, este refugio permite una **selección personalizada**. Los adoptantes pueden elegir el animal que mejor se adapte a su entorno tras consultar su ficha, historial y características.
3. **Sostenibilidad mediante Socios:** El refugio cuenta con una comunidad de socios que garantizan la viabilidad económica del centro. El sistema debe gestionar de forma precisa los datos de estos colaboradores, la emisión de cuotas anuales y el seguimiento de los pagos para evitar la morosidad.

### 2.2 Requisitos Funcionales (RF)

#### Bloque A: Gestión de Animales y Catálogo

- **RF01. Alta de Animales:** Registro de nuevos ingresos con datos de origen, especie, raza y características físicas. El sistema registra automáticamente la fecha de llegada.
- **RF02. Gestión de Catálogo de Especies y Razas:** Capacidad para dar de alta nuevas especies (ej. animales exóticos) y sus razas asociadas, manteniendo la integridad del sistema.
- **RF03. Consulta y Modificación de Ficha:** Visualización de los datos completos de un animal y la edición de sus características (carácter, color, etc.).
- **RF04. Búsqueda Multicriterio:** Localización de animales en el inventario mediante filtros por especie, raza, color, sexo o estado de disponibilidad.

#### Bloque B: Control Sanitario y Vacunación

- **RF05. Protocolo de Vacunación Automático:** Al dar de alta un animal, el sistema debe insertar automáticamente en su historial médico las vacunas marcadas como "esenciales" para su especie particular.
- **RF06. Suministro de Dosis Manuales:** Registro de nuevas dosis administradas en cualquier momento (refuerzos de esenciales o vacunas no esenciales), vinculándolas al catálogo de vacunas existente.
- **RF07. Consulta de Historial Clínico:** Obtención de la lista completa de dosis administradas a un animal, detallando fecha y tipo de vacuna.

#### **Bloque C: Proceso de Adopción**

- **RF08. Adopción Seleccionada:** Registro del proceso de adopción de un animal específico identificado por su ID único a elección del adoptante.
- **RF09. Cierre de Expediente de Adopción:** Tras la elección, el sistema debe registrar la fecha de salida y actualizar el estado del animal para que deje de aparecer como disponible en las búsquedas.

#### **Bloque D: Gestión de Socios y Economía**

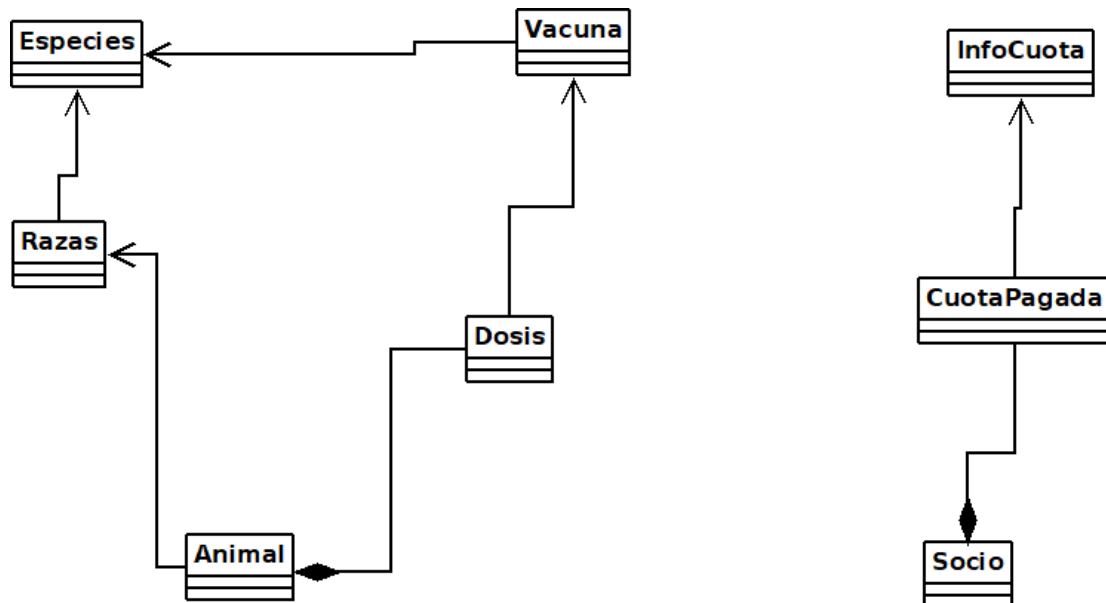
- **RF10. Mantenimiento de Socios (ABM):** Alta, baja y modificación de los datos de los socios (DNI, contacto, dirección).
- **RF11. Gestión de Cuotas Anuales:** Creación de ejercicios fiscales con importes definidos. El sistema debe asignar automáticamente estas cuotas a todos los socios activos mediante procesos en segundo plano.
- **RF12. Control de Pagos y Morosidad:** Registro de la recepción de pagos y generación de listados de socios con cuotas pendientes.

### **2.3 Requisitos No Funcionales (RNF)**

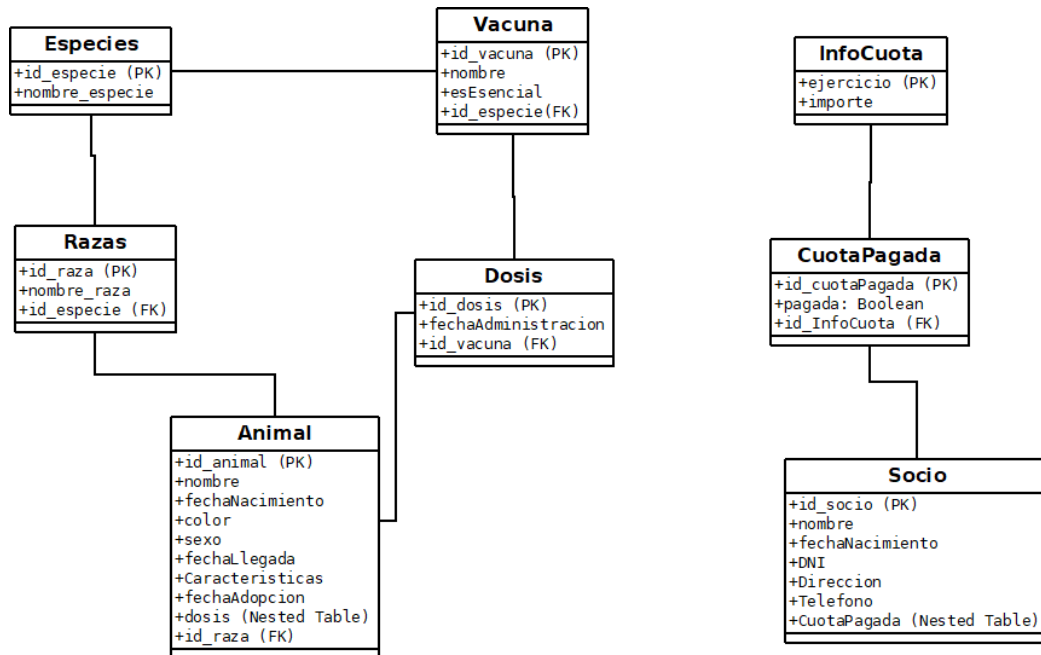
- **RNF01. Modelo Objeto-Relacional:** Los datos deben persistir mediante el uso de tipos de objeto (OBJECT TYPES) y tablas de objetos, aprovechando las ventajas de la orientación a objetos en la base de datos.
- **RNF02. Atributos Multievaluados:** El historial de vacunas y las cuotas de los socios deben almacenarse mediante **colecciones anidadas** (NESTED TABLES) para optimizar la cohesión de la información.
- **RNF03. Automatización mediante Triggers:** El sistema debe usar disparadores para garantizar que la lógica crítica (como la vacunación obligatoria al ingresar) se ejecute sin intervención manual, asegurando la integridad sanitaria.

- **RNF04. Encapsulación de Lógica:** Toda la operativa debe estar contenida en un **Paquete de PL/SQL**, proporcionando una interfaz limpia y segura para aplicaciones externas (como Python) y ocultando la complejidad del esquema.
- **RNF05. Integridad Referencial de Objetos:** Uso de punteros (REF) con alcance definido (SCOPE) para vincular dosis con vacunas y socios con sus cuotas, garantizando que no existan referencias huérfanas.
- **RNF06. Usabilidad y Rendimiento:** El acceso a los datos de un animal y su historial debe ser eficiente, minimizando el número de operaciones de unión (*joins*) mediante el uso de la estructura de objetos.

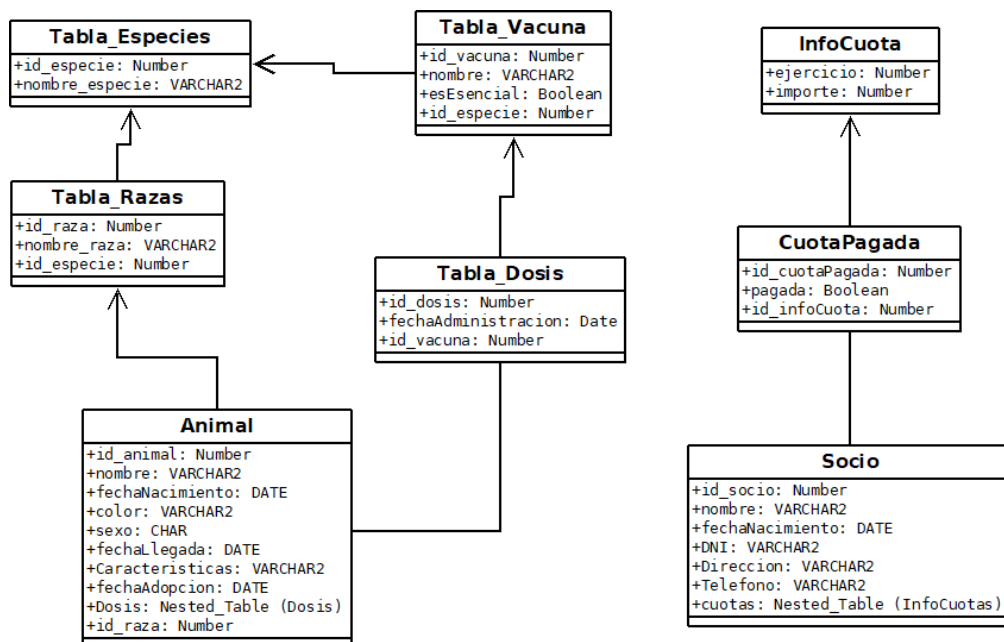
### 3. Diagrama de clases persistentes.



## 4. Esquema lógico específico O/R.



## 5. Diseño Físico.



## 6. Disparadores y paquete

Para garantizar la integridad de los datos y centralizar las operaciones del refugio, se ha implementado una capa de lógica programada en PL/SQL. Esta se divide en un paquete global de funciones y disparadores automáticos.

### 6.1 Paquete funcionesRefugio

El paquete funcionesRefugio actúa como la API principal del sistema. Su objetivo es encapsular las operaciones de inserción, actualización y consulta, asegurando que todas las transacciones sigan las mismas reglas y manejen las excepciones de forma controlada.

| Categoría           | Función                 | Descripción                                                              |
|---------------------|-------------------------|--------------------------------------------------------------------------|
| Gestión de Animales | <b>insertarAnimal</b>   | Registra un nuevo animal inicializando su lista de vacunas.              |
|                     | <b>actualizarAnimal</b> | Modifica los datos de un animal existente.                               |
|                     | <b>edadAnimal</b>       | Llama al método miembro del tipo de objeto para calcular la edad actual. |
|                     | <b>borrarAnimal</b>     | Elimina un registro de la tabla de animales.                             |
|                     | <b>adoptarAnimal</b>    | Marca la fecha de adopción del animal con la fecha actual del sistema.   |
| Salud y Vacunas     | <b>crearVacuna</b>      | Registra una nueva vacuna asociada a una especie.                        |
|                     | <b>suministrarDosis</b> | Inserta un objeto Tipo_Dosis en la colección anidada del animal.         |



| Categoría       | Función                       | Descripción                                                              |
|-----------------|-------------------------------|--------------------------------------------------------------------------|
| Socios y Cuotas | <b>insertarSocio</b>          | Registra un nuevo socio con su tabla de cuotas vacía.                    |
|                 | <b>insertarCuota</b>          | Define el importe de la cuota para un ejercicio económico concreto.      |
|                 | <b>asignarCuotaSocio</b>      | Vincula manualmente una cuota a un socio específico.                     |
| Configuración   | <b>obtenerRazasPorEspecie</b> | Devuelve una colección con los nombres de las razas de una especie dada. |
|                 | <b>insertarEspecie/Raza</b>   | Funciones auxiliares para el mantenimiento de tablas maestras.           |

## 6.2 Disparadores (Triggers)

Los disparadores se han diseñado para automatizar procesos críticos que no deben depender de la intervención manual del usuario.

### A. Trigger\_SuministrarEsenciales

- **Evento:** BEFORE INSERT ON Tabla\_Animal.
- **Propósito:** Automatizar el protocolo sanitario inicial.
- **Lógica:** Cuando un animal entra al refugio, el disparador busca en Tabla\_Vacuna todas aquellas vacunas marcadas como "Esenciales" (esEsencial = 'S') que correspondan a la especie del animal. Automáticamente, inserta estas vacunas en el historial del animal con la fecha de hoy.

### B. Trigger\_AsignarCuotas

- **Evento:** FOR INSERT ON Tabla\_InfoCuota (Compound Trigger).
- **Propósito:** Automatizar la facturación anual.
- **Lógica:** Al ser un disparador compuesto, gestiona la lógica en dos pasos para evitar errores de tablas mutantes:
  1. **After Each Row:** Captura el año del nuevo ejercicio creado.

2. **After Statement:** Una vez creada la cuota anual (ej. Ejercicio 2024), recorre todos los socios registrados y les asigna automáticamente dicha cuota en su historial personal con el estado de pago en 'N' (No pagada).

## 7. Conclusiones

Una vez finalizado el desarrollo del sistema de gestión para el refugio, estas son las principales conclusiones sobre lo que hemos conseguido con esta solución:

- **Un modelo que entiende la realidad (O/R):** Usar el modelo Objeto-Relacional ha sido todo un acierto. En lugar de tener la información "desparramada" en decenas de tablas inconexas, hemos logrado que los datos médicos (vacunas) y económicos (cuotas) viajen pegados a su dueño (el animal o el socio). Esto no solo hace que la base de datos sea más intuitiva y fiel a la vida real, sino que también la hace mucho más rápida al evitar cruces de datos innecesarios.
- **Tranquilidad veterinaria gracias a la automatización:** Quizás uno de los puntos más valiosos es haber logrado que la base de datos sea "inteligente". Gracias a los *triggers*, el refugio tiene la seguridad de que, en cuanto entra un animal, el sistema le asigna sus vacunas esenciales automáticamente. Esto elimina el miedo al error humano o al olvido, garantizando que todos los animales reciban sus cuidados básicos desde el primer minuto.
- **Datos con "vida propia":** Al crear el paquete funcionesRefugio y meter lógica directamente en los objetos (como el cálculo automático de la edad), hemos conseguido que los datos no sean simples textos estáticos. Ahora, el dato sabe "comportarse". Esto facilita muchísimo el trabajo si en el futuro queremos conectar el sistema con una web o una app móvil, ya que las reglas siempre serán las mismas y el código será más fácil de ampliar.
- **Conexiones seguras y sin duplicados:** Mediante el uso de referencias lógicas (REF), hemos creado un sistema de "punteros" muy robusto. Esto nos permite conectar un animal con su vacuna o un socio con su cuota de forma limpia, sin necesidad de repetir información. Es una forma elegante de asegurar que todo esté en su sitio y que no existan datos "huérfanos" o erróneos.

- **Orden administrativo para la supervivencia del centro:** Por último, el sistema no se queda solo en lo veterinario. Hemos construido una estructura que da claridad a las cuentas del refugio. Al tener las cuotas de los socios integradas en sus perfiles, el control de los pagos y la detección de la morosidad son inmediatos. En un entorno donde los recursos son limitados, esta visibilidad es fundamental para que el refugio sea sostenible a largo plazo.

En definitiva, este proyecto es la prueba de que, cuando combinamos la potencia de Oracle con la flexibilidad de los objetos, la tecnología deja de ser una barrera y se convierte en una herramienta que se adapta perfectamente a las necesidades reales del refugio.

## 8. Referencias bibliográficas.

Modelos de datos: Modelo Conceptual, Físico y Lógico (<https://www.tecnologias-informacion.com/modelos-datos.html>)

Disparadores Compuestos (Compound Triggers)  
(<https://profesionghh.blogspot.com/2015/01/disparadores-compuestos-compound.html>)

Triggers Documentacion ([https://docs.oracle.com/html/B28370\\_05/triggers.htm](https://docs.oracle.com/html/B28370_05/triggers.htm))

API QT (<https://doc.qt.io/qt-6/qtdesigner-manual.html>)

API QT Ingles (<https://doc.qt.io/qt-6/es/qtdesigner-index.html>)

## 9. Manual de instalación

1. Instalación de las librerías necesarias a través de:  
**pip install -r requirements.txt**
2. Instalación de la base de datos a través de la ejecución en Oracle  
del script: **main.sql**
3. Establecer los campos para la conexión dentro del fichero  
**conexion.ini**
4. Ejecutar el proyecto (main.py)